



FUNDACIÓN ESCUELA
TECNOLOGICA DE NEIVA
JESÚS OVIEDO PÉREZ

Taller de Laboratorio

Autor:

Andres Ricardo Rodriguez

Fundación Escuela Tecnológica de Neiva “FET” Jesus Oviedo Pérez

Docente: Juan Carlos Polanía Cortes

Ingeniería de Software – Optativa IV

Contenido

1. Introducción	3
2. Objetivos.....	3
2.1. Objetivo General	3
2.2. Objetivos Específicos.....	3
3. Instructivo de Reproducción y Montaje.....	4
4. Creación de la Base de Datos y la Tabla	6
4. Código Fuente de la API REST	7
5. Evidencias de Verificación y Pruebas.....	8
6. Declaración de Uso de Inteligencia Artificial.....	10
7. Conclusión	11

1. Introducción

En este laboratorio se implementó una arquitectura backend desacoplada integrando Docker y un servicio API REST. En primera instancia, se desplegó un motor de base de datos relacional MySQL dentro de un contenedor aislado, configurando un volumen para la persistencia de datos y mapeo de puertos para la comunicación local. Posteriormente, se desarrolló una API REST utilizando Python con FastAPI para conectarse al contenedor y gestionar las operaciones CRUD (Crear, Leer, Actualizar y Eliminar) sobre una tabla relacional.

2. Objetivos

2.1. Objetivo General

Desplegar una base de datos MySQL en un contenedor Docker y construir una API REST propia y consumirla.

2.2. Objetivos Específicos

- Configurar y ejecutar un contenedor de MySQL utilizando Docker.
- Diseñar e implementar el script SQL para la creación de una base de datos y una tabla relacional.
- Desarrollar una API REST en Python (FastAPI) que exponga los endpoints necesarios para las operaciones POST, GET, PUT y DELETE.

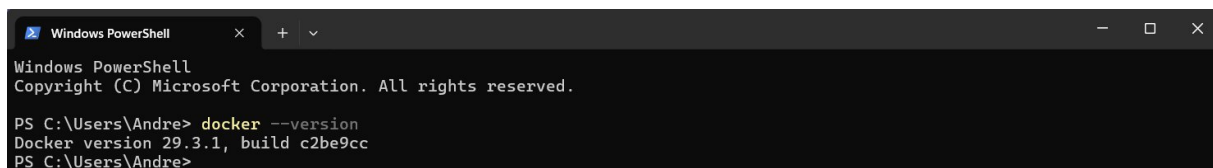
3. Instructivo de Reproducción y Montaje

Para reproducir este proyecto en un entorno local hay que seguir los siguientes pasos:

Paso 1: Verificación de Docker

Para este primer paso, con Docker instalado y en ejecución en el host, hay que abrir una terminal (PowerShell o CMD) y ejecutar el comando de verificación:

```
docker --version
```



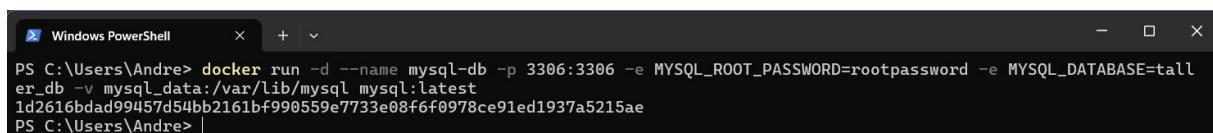
```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Andre> docker --version
Docker version 29.3.1, build c2be9cc
PS C:\Users\Andre>
```

Paso 2: Despliegue del Contenedor MySQL

Con el servicio de Docker activo, se ejecuta el siguiente comando en la terminal para levantar el motor de base de datos MySQL en segundo plano, exponiendo el puerto 3306 y configurando un volumen para garantizar la persistencia de la información:

```
docker run -d --name mysql-db -p 3306:3306 -e MYSQL_ROOT_PASSWORD=rootpassword
-e MYSQL_DATABASE=taller_db -v mysql_data:/var/lib/mysql mysql:latest
```



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Andre> docker run -d --name mysql-db -p 3306:3306 -e MYSQL_ROOT_PASSWORD=rootpassword -e MYSQL_DATABASE=taller_db -v mysql_data:/var/lib/mysql mysql:latest
1d2616bdad99457d54bb2161bf990559e7733e08f6f0978ce91ed1937a5215ae
PS C:\Users\Andre>
```

- **Detalle de la configuración:**

- `-d`: Inicia el contenedor en segundo plano (detached mode).
- `--name mysql-db`: Asigna la etiqueta identificadora al contenedor.
- `-p 3306:3306`: Publica el puerto 3306 para habilitar la comunicación entre el equipo anfitrión y MySQL.
- `-e MYSQL_ROOT_PASSWORD=rootpassword`: Establece la contraseña de administración del usuario root.
- `-e MYSQL_DATABASE=taller_db`: Define la base de datos inicial.
- `-v mysql_data:/var/lib/mysql`: Asocia el volumen `mysql_data` al directorio de almacenamiento de MySQL, asegurando que los datos no se borren al detener el contenedor.

Posteriormente, se comprueba el estado del contenedor ejecutando:

```
docker ps
```

```
Windows PowerShell
PS C:\Users\Andre> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
1d2616bdad99   mysql:latest   "docker-entrypoint.s..." 26 seconds ago Up 26 seconds  0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp
PS C:\Users\Andre>
```

Paso 3: Ejecución de la API REST

Se realiza la instalación de las dependencias necesarias en la máquina anfitriona mediante el comando:

```
py -m pip install fastapi uvicorn mysql-connector-python
```

```
C:\Windows\System32\cmd.e
Microsoft Windows [Versión 10.0.26200.9457]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Andre\Downloads\code>py -m pip install fastapi uvicorn mysql-connector-python
Requirement already satisfied: fastapi in C:\Users\Andre\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (0.14.1)
Requirement already satisfied: uvicorn in C:\Users\Andre\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (0.53.0)
Requirement already satisfied: mysql-connector-python in C:\Users\Andre\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (26.7.0)
Requirement already satisfied: starlette>=0.46.0 in C:\Users\Andre\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from fastapi) (1.6.0)
Requirement already satisfied: pydantic>=2.9.0 in C:\Users\Andre\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from fastapi) (2.13.5)
Requirement already satisfied: typing-extensions>=4.8.0 in C:\Users\Andre\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from fastapi) (4.16.0)
Requirement already satisfied: typing-inspection>=0.4.2 in C:\Users\Andre\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from fastapi) (0.4.4)
Requirement already satisfied: annotated-doc>=0.0.2 in C:\Users\Andre\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from fastapi) (0.0.5)
Requirement already satisfied: click>=7.0 in C:\Users\Andre\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from uvicorn) (8.5.0)
Requirement already satisfied: h11>=0.8 in C:\Users\Andre\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from uvicorn) (0.16.0)
Requirement already satisfied: annotated-types>=0.6.0 in C:\Users\Andre\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from pydantic>=2.9.0->fastapi) (0.8.0)
Requirement already satisfied: pydantic-core==2.46.5 in C:\Users\Andre\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from pydantic>=2.9.0->fastapi) (2.46.5)
Requirement already satisfied: anyio<5,>=3.6.2 in C:\Users\Andre\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from starlette>=0.46.0->fastapi) (4.15.1)
```

Finalmente, se inicia el servidor de desarrollo backend ejecutando:

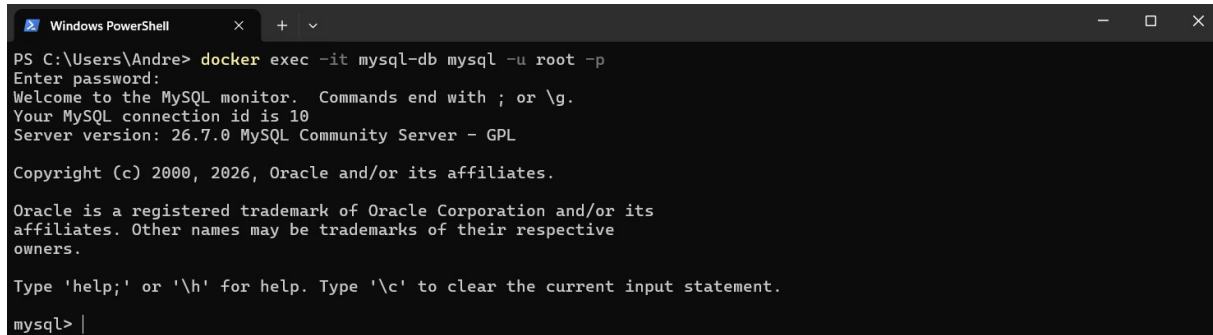
```
py -m uvicorn main:app --reload
```

```
C:\Windows\System32\cmd.e
C:\Users\Andre\Downloads\code>py -m uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['C:\\Users\\Andre\\Downloads\\code']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [19916] using StatReload
ERROR: Error loading ASGI app. Could not import module "main".
```

4. Creación de la Base de Datos y la Tabla

Para la definición del modelo de datos no se requirió la ejecución de un archivo externo. En su lugar, se accedió de manera directa a la consola interactiva de MySQL dentro del contenedor Docker en ejecución mediante el siguiente comando:

```
docker exec -it mysql-db mysql -u root -p
```



```
Windows PowerShell
PS C:\Users\Andre> docker exec -it mysql-db mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 10
Server version: 26.7.0 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> |
```

Una vez autenticado en la consola de MySQL, se ejecutaron las sentencias SQL para estructurar la base de datos y la tabla productos:

```
CREATE DATABASE IF NOT EXISTS taller_db;

USE taller_db;

CREATE TABLE productos (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  precio DECIMAL(10, 2) NOT NULL,
  disponible BOOLEAN NOT NULL DEFAULT TRUE,
  fecha_creacion DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

Estrategia de diseño de la tabla:

- id: Clave primaria autoincremental para la identificación única de cada registro.
- nombre: Campo de texto (VARCHAR) obligatorio (NOT NULL) que almacena el nombre del producto.
- precio: Campo numérico decimal (DECIMAL) obligatorio (NOT NULL) para el valor monetario.
- disponible: Campo booleano (BOOLEAN) obligatorio para indicar el estado del registro.
- fecha_creacion: Campo temporal (DATETIME) con valor por defecto de la fecha actual del servidor.

4. Código Fuente de la API REST

Se desarrolló el servicio web backend en Python haciendo uso del framework FastAPI y el conector mysql-connector-python para gestionar la persistencia en el contenedor. El código fuente se estructuró en el archivo main.py, garantizando la devolución de respuestas con encabezado Content-Type: application/json y el esquema estricto solicitado (status, message, data): (en el archivo).

5. Evidencias de Verificación y Pruebas

Para comprobar el correcto funcionamiento de la API REST y el ciclo de vida de los datos, se estructuró una suite de 8 peticiones en el cliente REST Insomnia. Las pruebas se ejecutaron en el siguiente orden secuencial:

Petición 1: Crear un registro (POST)

The screenshot shows a POST request to `http://127.0.0.1:8000/productos` with a status of **201 Created**, taking 323 ms and returning 121 B. The request body is a JSON object:

```
{ "nombre": "Teclado Mecánico", "precio": 45.50, "disponible": true }
```

. The response body is a JSON object:

```
{ "status": true, "message": "Registro creado", "data": { "id": 52, "nombre": "Teclado Mecánico", "precio": 45.5, "disponible": true } }
```

Petición 2: Listar todos los registros (GET)

The screenshot shows a GET request to `http://127.0.0.1:8000/productos` with a status of **200 OK**, taking 55 ms and returning 165 B. The response body is a JSON object:

```
{ "status": true, "message": "Registros consultados", "data": [ { "id": 52, "nombre": "Teclado Mecánico", "precio": 45.5, "disponible": 1, "fecha_creacion": "2026-09-17T22:55:16" } ] }
```

Petición 3: Consultar registro específico por ID (GET)

The screenshot shows a GET request to `http://127.0.0.1:8000/productos/52` with a status of **200 OK**, taking 37 ms and returning 161 B. The response body is a JSON object:

```
{ "status": true, "message": "Registro consultado", "data": { "id": 52, "nombre": "Teclado Mecánico", "precio": 45.5, "disponible": 1, "fecha_creacion": "2026-09-17T22:55:16" } }
```

Petición 4: Actualizar un registro existente (PUT)

The screenshot shows a REST client interface with a PUT request to `http://127.0.0.1:8000/productos/52`. The request body is a JSON object: `{ "nombre": "Teclado RGB", "precio": 50.00, "disponible": false }`. The response is a 200 OK status with a response body: `{ "status": true, "message": "Registro actualizado", "data": { "id": 52, "nombre": "Teclado RGB", "precio": 50.0, "disponible": false } }`.

```
PUT http://127.0.0.1:8000/productos/52
{
  "nombre": "Teclado RGB",
  "precio": 50.00,
  "disponible": false
}
```

```
{
  "status": true,
  "message": "Registro actualizado",
  "data": {
    "id": 52,
    "nombre": "Teclado RGB",
    "precio": 50.0,
    "disponible": false
  }
}
```

Petición 5: Re-consultar el registro modificado (GET)

The screenshot shows a REST client interface with a GET request to `http://127.0.0.1:8000/productos/52`. The response is a 200 OK status with a response body: `{ "status": true, "message": "Registro consultado", "data": { "id": 52, "nombre": "Teclado RGB", "precio": 50.0, "disponible": 0, "fecha_creacion": "2026-09-17T22:55:16" } }`.

```
GET http://127.0.0.1:8000/productos/52
```

```
{
  "status": true,
  "message": "Registro consultado",
  "data": {
    "id": 52,
    "nombre": "Teclado RGB",
    "precio": 50.0,
    "disponible": 0,
    "fecha_creacion": "2026-09-17T22:55:16"
  }
}
```

Petición 6: Eliminar el registro por ID (DELETE)

The screenshot shows a REST client interface with a DELETE request to `http://127.0.0.1:8000/productos/52`. The response is a 200 OK status with a response body: `{ "status": true, "message": "Registro eliminado" }`.

```
DELETE http://127.0.0.1:8000/productos/52
```

```
{
  "status": true,
  "message": "Registro eliminado"
}
```

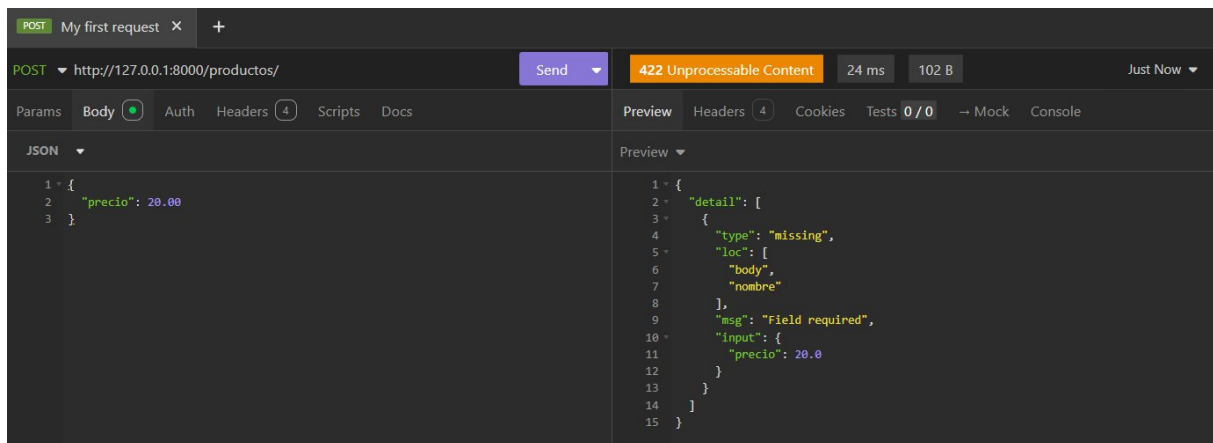
Petición 7: Manejo de error - Consultar registro eliminado (GET)

The screenshot shows a REST client interface with a GET request to `http://127.0.0.1:8000/productos/52`. The response is a 404 Not Found status with a response body: `{ "status": false, "message": "Registro no encontrado" }`.

```
GET http://127.0.0.1:8000/productos/52
```

```
{
  "status": false,
  "message": "Registro no encontrado"
}
```

Petición 8: Manejo de error - Omisión de campo obligatorio (POST)



6. Declaración de Uso de Inteligencia Artificial

De acuerdo con las directrices de la guía de laboratorio, se declara el uso de herramientas de inteligencia artificial durante el desarrollo del proyecto bajo las siguientes condiciones:

- Herramienta utilizada: Gemini (Google AI).
- Finalidad: Asistencia técnica en la estructuración de la arquitectura del proyecto, la sintaxis del framework FastAPI en Python para el manejo de respuestas HTTP y la redacción del presente informe técnico.
- Modificaciones realizadas: Se adaptó el modelo de datos para reflejar la tabla productos con sus correspondientes tipos de datos, se configuró la lógica de conexión directa con la base de datos MySQL en el contenedor Docker y se validó manualmente cada una de las peticiones en el cliente Insomnia para garantizar el cumplimiento de los requerimientos.

7. Conclusión

El desarrollo de este laboratorio evidenció la eficiencia de integrar un motor relacional MySQL contenedorizado en Docker junto a una API REST construida en FastAPI, demostrando que la separación de responsabilidades y la persistencia mediante volúmenes garantizan un entorno backend aislado, portátil y escalable. Asimismo, la correcta implementación de las operaciones CRUD, el manejo riguroso de códigos de estado HTTP y la estandarización de las respuestas JSON permitieron validar un ciclo de vida de datos robusto y preparado para la integración con cualquier cliente o aplicación externa.